3.10.2、直方图均衡

[docs.opencv.org]
https://docs.opencv.org/4.x/d5/daf/tutorial_py_histogram_equalization.html

1. 为什么需要直方图均衡化?

很多图像的像素值会集中在某个狭窄范围：

- 太亮的照片 → 像素大多集中在高值区域（右边）
- 太暗的照片 → 像素大多集中在低值区域（左边）
  直方图均衡化会把这些集中的像素值拉伸到整个 0~255 范围，让对比度显著提升，细节更清晰。简单来说：它让直方图变得更“均匀”，让图像从“灰蒙蒙”变得更有层次。注意：它对全局对比度有效，但有时会让某些区域过亮或丢失细节（后面会介绍 CLAHE 来解决）。

2. 理论基础（Numpy 实现）

先用 Numpy 手动实现一遍，帮助你理解原理。核心步骤：

- 计算直方图 hist
- 计算累计分布函数（CDF）
- 把 CDF 归一化到 0~255，得到查找表（LUT）
- 用查找表映射原图像

```
import cv2 as cv
import numpy as np
from matplotlib import pyplot as plt

img = cv.imread('wiki.jpg', cv.IMREAD_GRAYSCALE)  # 换成你的图像路径
assert img is not None, "文件读取失败，请检查路径"

# 计算直方图和 CDF
hist, bins = np.histogram(img.flatten(), 256, [0, 256])
cdf = hist.cumsum()                          # 累计和
cdf_normalized = cdf * hist.max() / cdf.max()   # 归一化显示用

# 绘制原始直方图 + CDF
plt.plot(cdf_normalized, color='b', label='CDF')
plt.hist(img.flatten(), 256, [0, 256], color='r', label='Histogram')
plt.xlim([0, 256])
plt.legend()
```

```
plt.title('原始图像直方图与 CDF')
plt.show()
```

你会看到原始直方图集中在某个区域（例如右边亮区）。均衡化公式实现（关键部分）：

```
# 掩码处理，避免除以0
cdf_m = np.ma.masked_equal(cdf, 0)
cdf_m = (cdf_m - cdf_m.min()) * 255 / (cdf_m.max() - cdf_m.min())
cdf = np.ma.filled(cdf_m, 0).astype('uint8')

# 应用变换（最简单的一行）
img2 = cdf[img]   # 用查找表映射每个像素

# 显示对比
res = np.hstack((img, img2))   # 左右拼接
cv.imwrite('equalized.png', res)
plt.imshow(res, 'gray')
plt.title('左:原始  右:均衡化后')
plt.show()
```

均衡化后，直方图会覆盖几乎整个 0~255 范围，对比度明显变好。

3. OpenCV 内置函数（推荐使用）

OpenCV 提供超级简单的函数：cv.equalizeHist()

```
img = cv.imread('wiki.jpg', cv.IMREAD_GRAYSCALE)
assert img is not None, "文件读取失败"

equ = cv.equalizeHist(img)          # 一行搞定直方图均衡化

# 左右对比显示
res = np.hstack((img, equ))
cv.imshow('Equalization', res)
cv.waitKey(0)
cv.destroyAllWindows()

# 保存结果
cv.imwrite('res.png', res)
```

优点：非常快，只支持灰度图（单通道）。
缺点：这是全局均衡化。如果图像中同时有很亮和很暗的区域，可能会导致某些部分过曝或过暗，丢失细节（例如人脸区域变太亮）。

4. CLAHE —对比度受限的自适应直方图均衡化（强烈推荐）

这是对全局均衡化的改进版本，效果更好，尤其适合真实照片。原理：

- 把图像分成很多小块（tiles，默认 8x8）
- 对每个小块分别做直方图均衡化（局部处理）
- 加入 对比度限制（clipLimit），防止噪声被过度放大
- 块与块之间用双线性插值平滑过渡

```
import cv2 as cv

img = cv.imread('tsukuba_l.png', cv.IMREAD_GRAYSCALE)  # 换成你的图像
assert img is not None, "文件读取失败"

# 创建 CLAHE 对象（可以调整参数）
clahe = cv.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))

cl1 = clahe.apply(img)      # 应用 CLAHE

# 保存或显示
cv.imwrite('clahe_2.jpg', cl1)

# 对比显示（原始 vs 全局均衡化 vs CLAHE）
equ = cv.equalizeHist(img)
res = np.hstack((img, equ, cl1))
cv.imshow('Original | Global Equalize | CLAHE', res)
cv.waitKey(0)
```

参数说明：

- clipLimit：对比度限制，默认 40.0。值越大 → 对比度越强（但可能引入噪声）；建议从 2.0 开始尝试。
- tileGridSize：小块大小，默认 (8,8)。值越大 → 越接近全局均衡化；值越小 → 局部细节越明显。
  实际效果：
- 全局均衡化（equalizeHist）常把背景变好，但人脸或重要区域可能过亮。
- CLAHE 能同时提升背景和局部细节，效果更自然。